

Containerization_and_DevOps

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

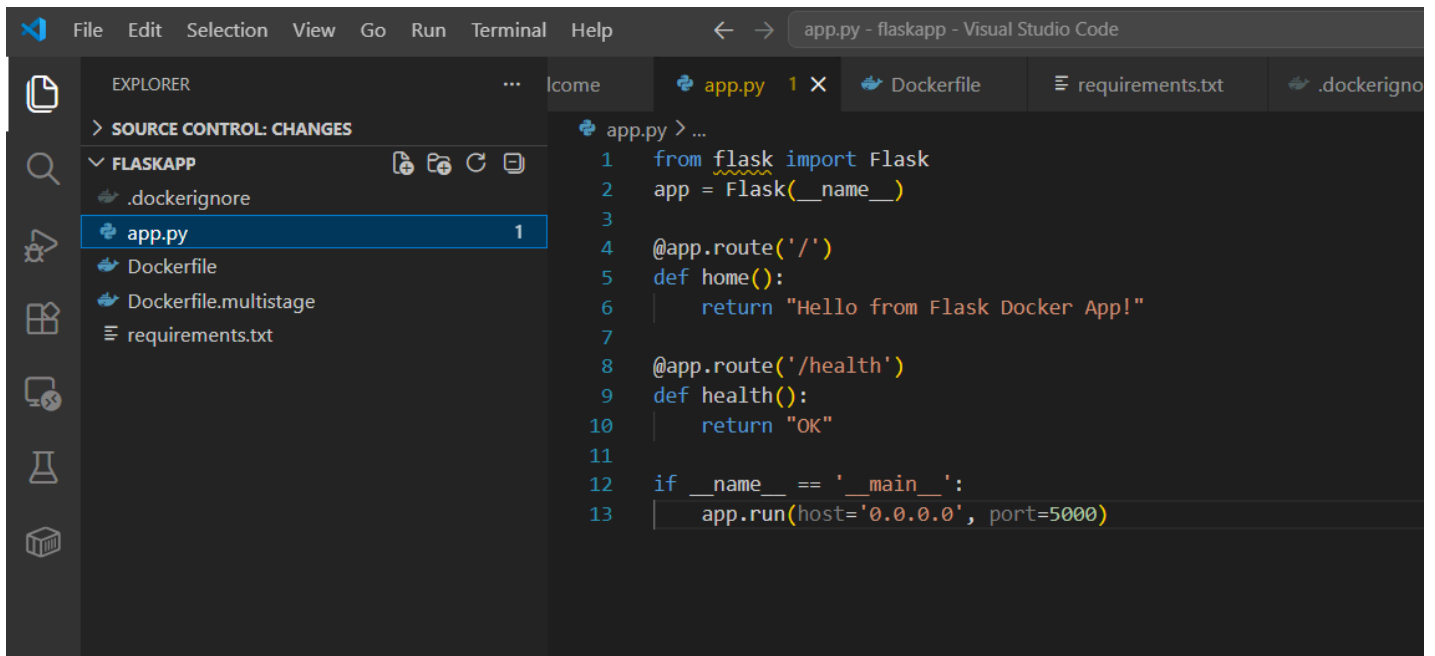
Step-1: Create a Directory for Python application

```
mkdir my-flask-app  
cd my-flask-app
```

```
C:\Users\ASUS>wsl  
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir flaskapp  
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cd flaskapp
```

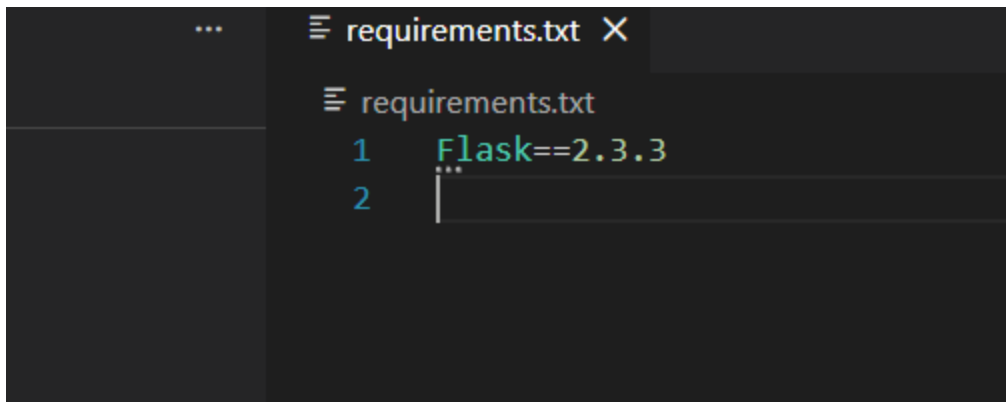
Step-2:- Create an **** app.py ****

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```



Step-3:- Create **** requirements.txt ****

Flask==2.3.3



Step-4: Create ****Dockerfile****

```

# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

```

```
# Copy application code
```

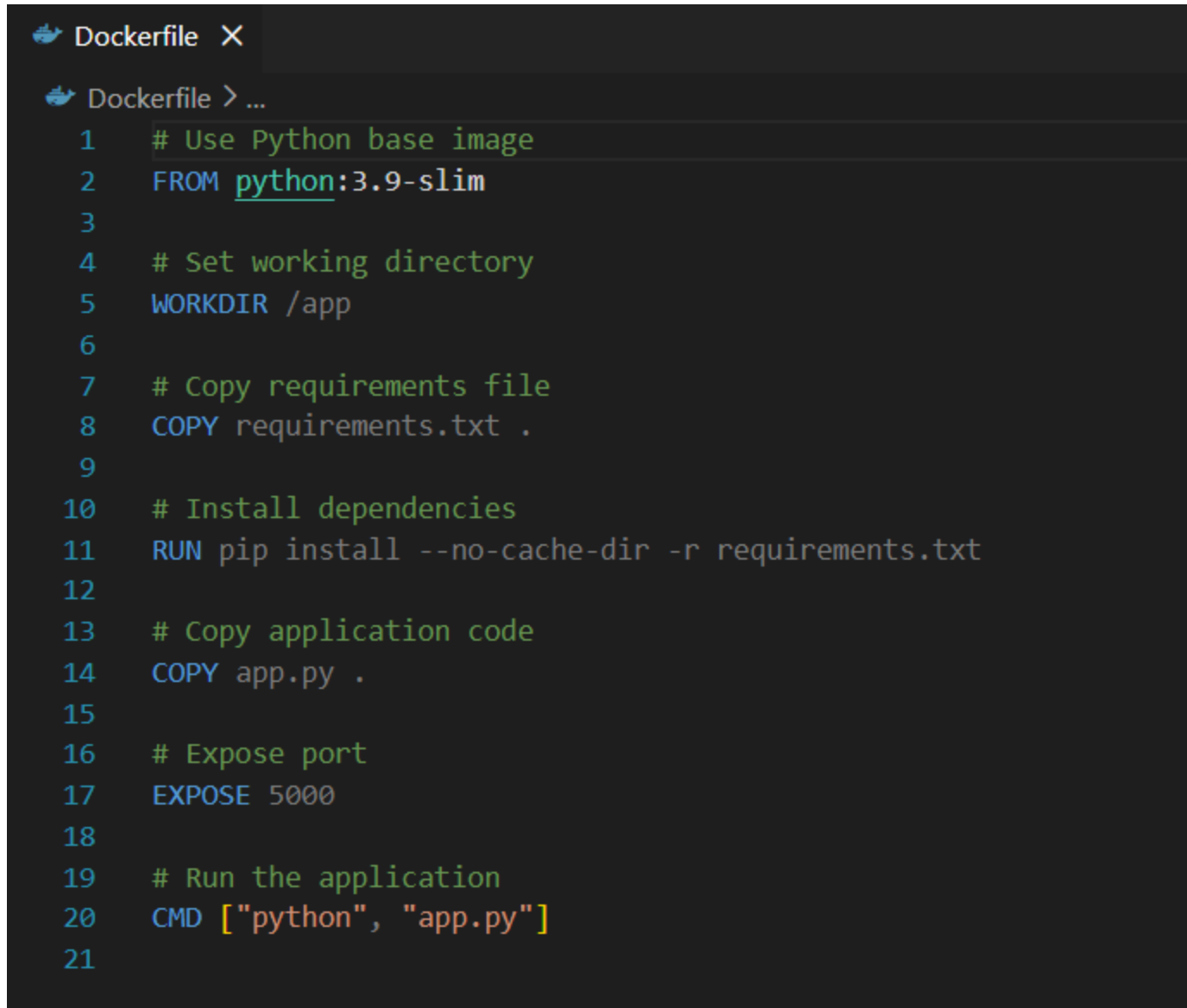
```
COPY app.py .
```

```
# Expose port
```

```
EXPOSE 5000
```

```
# Run the application
```

```
CMD ["python", "app.py"]
```

A screenshot of a Dockerfile editor window. The window has a title bar that says "Dockerfile X". Below the title bar, there is a breadcrumb "Dockerfile > ...". The main area of the editor shows a Dockerfile with 21 lines of code. The code is as follows:

```
1  # Use Python base image
2  FROM python:3.9-slim
3
4  # Set working directory
5  WORKDIR /app
6
7  # Copy requirements file
8  COPY requirements.txt .
9
10 # Install dependencies
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copy application code
14 COPY app.py .
15
16 # Expose port
17 EXPOSE 5000
18
19 # Run the application
20 CMD ["python", "app.py"]
21
```

Step-5:- Create a **** .dockerignore :***

```
# Python files
```

```
__pycache__/
```

```
*.pyc
```

```
*.pyo
```

```
*.pyd
```

Environment files

.env
.venv
env/
venv/

IDE files

.vscode/
.idea/

Git files

.git/
.gitignore

OS files

.DS_Store
Thumbs.db

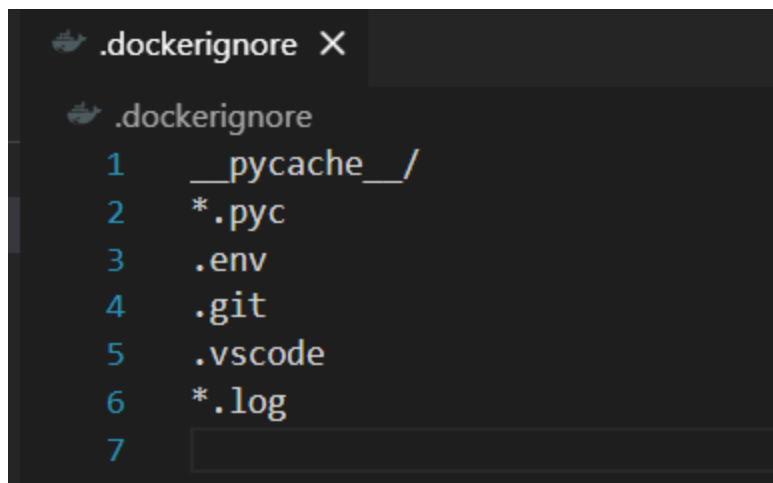
Logs

*.log
logs/

Test files

tests/
test_*.py

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ touch .dockerignore
```



Why *.dockerignore* is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp .
[+] Building 14.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => sha256:ea56f685404adf81680322f152d2cfe62115b30dda481c2c450078315beb508 251B / 251B
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> => extracting sha256:ea56f685404adf81680322f152d2cfe62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 342B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee0bd23d034a23390c9f3024454640
=> => exporting attestation manifest sha256:610cb19896fd82c4704a3dcff20ddb1bbe6bbb3f1c3c607ad784a29e7e0b96c3
=> => exporting manifest list sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6f76c4b
=> => naming to docker.io/library/flaskapp:latest
=> => unpacking to docker.io/library/flaskapp:latest
```

Step-7:- Verify Image

```
docker images
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
flaskapp             latest             d05c6222eb8d       43 seconds ago     200MB
containerassgn-mybackend latest             c4739b79801f       2 days ago         123MB
containerassgn-mydatabase latest             8e67228ab04f       2 days ago         392MB
projectassignment-backend latest             c8b9f1cdc720       2 days ago         199MB
```

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp:1.0 .
docker tag flaskapp:latest flaskapp:v1.0
[+] Building 2.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee0bd23d034a23390c9f3024454640
=> => exporting attestation manifest sha256:6eb59ae04c1f806e473fff86d53204ac22e5d5fdc73c872cd31bc7b18c403c60
=> => exporting manifest list sha256:978afe0cc550cf6a71b0ea813d9fad93dd4b0deebe9784020ed1b2681a5e2ecb
=> => naming to docker.io/library/flaskapp:1.0
=> => unpacking to docker.io/library/flaskapp:1.0
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t aarohi26/flaskapp:1.0 .
[+] Building 0.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load .dockerignore
=> => transferring context: 93B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --only=production
=> CACHED [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e8cc5b63d64bc1ddd1ffa5203ebb21bfa58cf44af39be6c4ceede3132fc108fc
=> => exporting config sha256:2c71dcc281f7fb7b39847e817ce1d6d6e723e46a13b87fe30bad1c00b919d575
=> => exporting attestation manifest sha256:e5acb8210abd371bcc05f68bdafcb22b05030650bd9f68e8230a4a24337e0147
=> => exporting manifest list sha256:1d265ba80b1bcff8919bce5c8299578bf7e471033214fd916aa92df0998cd613
=> => naming to docker.io/aarohi26/flaskapp:1.0
=> => unpacking to docker.io/aarohi26/flaskapp:1.0
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-12: View Image Details

```
docker images
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
d05c6222eb8d	25 hours ago	CMD ["python" "app.py"]	0B	buildkit.dockerfile.v0

Step-13:- Show image history

```
docker history my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker inspect flaskapp
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-04-04T03:12:31.713516257Z",
    "Descriptor": {
      "digest": "sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": "",
    "GraphDriver": {
      "Data": null,
      "Name": "overlayfs"
    }
  }
]
```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker inspect flaskapp
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-04-04T03:12:31.713516257Z",
    "Descriptor": {
      "digest": "sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": "",
    "GraphDriver": {
      "Data": null,
      "Name": "overlayfs"
    }
  }
]
```

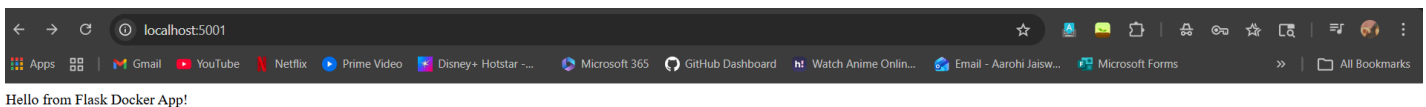
Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ curl http://localhost:5001
Hello from Flask Docker App!aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$
```

Step-16:- Test the application

```
curl http://localhost:5000
```



Step-17:- View running containers


```
docker ps
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker logs flask-container
docker stop flask-container
docker start flask-container
docker rm -f flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [04/Apr/2026 03:30:27] "GET / HTTP/1.1" 200 -
flask-container
flask-container
flask-container
```

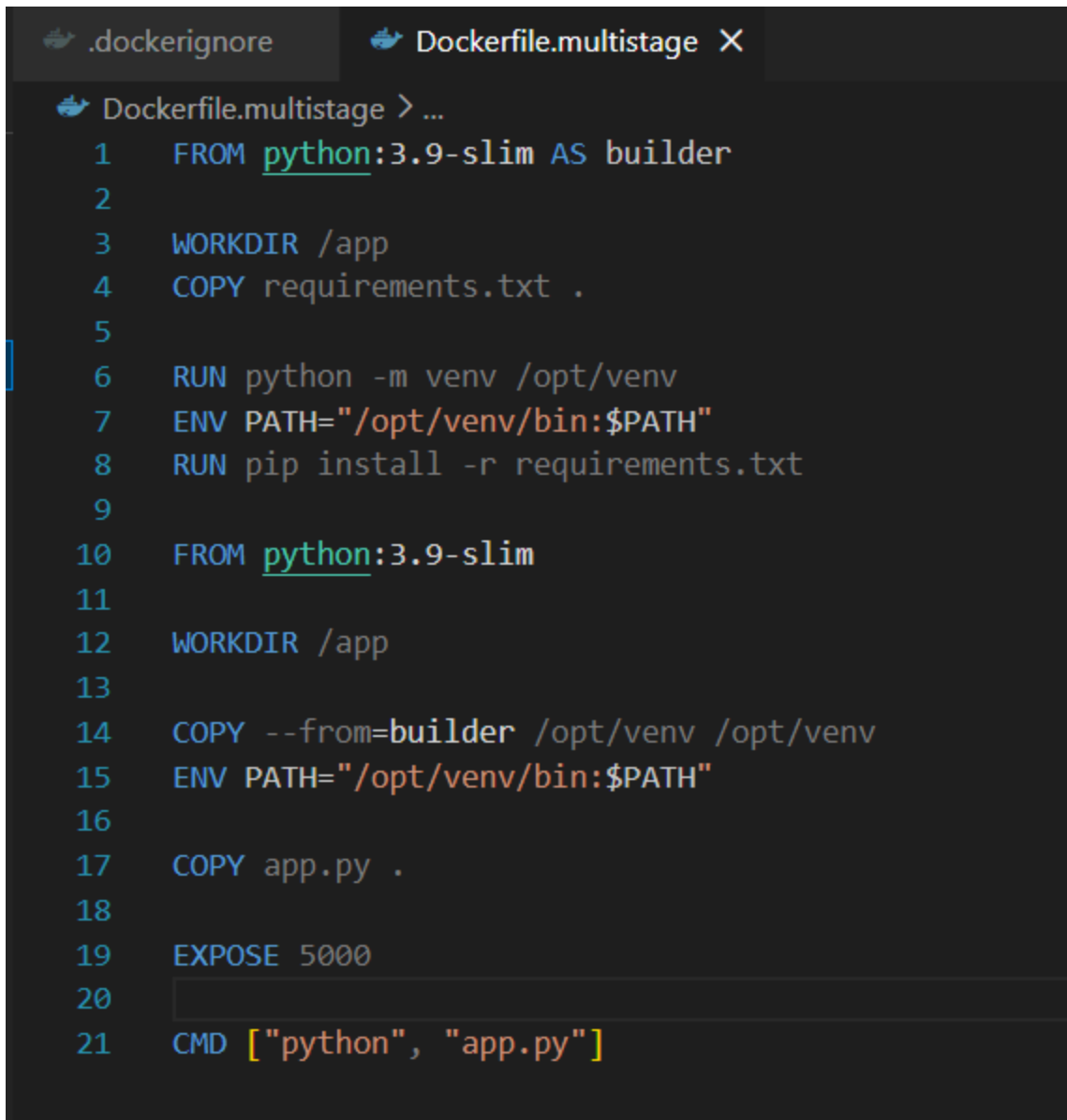
Step-18:- View container logs

```
docker logs flask-container
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ touch Dockerfile.multistage
```

Step-19:- Stop container

```
docker stop flask-container
```



```
.dockerignore Dockerfile.multistage X
Dockerfile.multistage > ...
1 FROM python:3.9-slim AS builder
2
3 WORKDIR /app
4 COPY requirements.txt .
5
6 RUN python -m venv /opt/venv
7 ENV PATH="/opt/venv/bin:$PATH"
8 RUN pip install -r requirements.txt
9
10 FROM python:3.9-slim
11
12 WORKDIR /app
13
14 COPY --from=builder /opt/venv /opt/venv
15 ENV PATH="/opt/venv/bin:$PATH"
16
17 COPY app.py .
18
19 EXPOSE 5000
20
21 CMD ["python", "app.py"]
```

Step-20:- Start stopped container

```
docker start flask-container
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp-regular .
docker build -f Dockerfile.multistage -t flaskapp-optimized .
[+] Building 1.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee8bd23d034a23390c9f3024454648
=> => exporting attestation manifest sha256:ccb44d69ee3508fbf57d9cc8b8bd47ad95e4ae49131eb491c1b11379a20f96aa1
=> => exporting manifest list sha256:cddc6afccf44a6dfdf560f41ef7a2bf80bb8597760f530fe8449b9771532c6c0
=> => naming to docker.io/library/flaskapp-regular:latest
=> => unpacking to docker.io/library/flaskapp-regular:latest
[+] Building 11.8s (12/12) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 398B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt
=> [builder 4/5] RUN python -m venv /opt/venv
=> [builder 5/5] RUN pip install -r requirements.txt
=> [stage-1 3/4] COPY --from=builder /opt/venv /opt/venv
=> [stage-1 4/4] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:82a48dddbc309115d20aa3589b1431c79f914ee0c2d43ddac6ff580f44439709
=> => exporting config sha256:8f4a58beae55e87d0f3485f1cd643bc6d8764be9e70a6e0e874541e7f1d7beca
=> => exporting attestation manifest sha256:2a065b6d8e7749b3e146b18e0f3375cf830595f60747b85917faf2cb869da5ff
=> => exporting manifest list sha256:93c15db1fc034b435626c52a5c466b60de4940c7df7cb570a961af3c1b049d5f
=> => naming to docker.io/library/flaskapp-optimized:latest
=> => unpacking to docker.io/library/flaskapp-optimized:latest

```

Step-21:- Remove container

```
docker rm flask-container
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 2.9s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 398B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 89B
=> [internal] load build context
=> => transferring context: 63B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale1
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale1
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt
=> CACHED [builder 4/5] RUN python -m venv /opt/venv
=> CACHED [builder 5/5] RUN pip install -r requirements.txt
=> CACHED [stage-1 3/4] COPY --from=builder /opt/venv /opt/venv
=> CACHED [stage-1 4/4] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:82a48dddbc309115d20aa3589b1431c79f914ee0c2d43ddac6ff580f44439709
=> => exporting config sha256:8f4a58beae55e87d0f3485f1cd643bc6d8764be9e70a6e0e874541e7f1d7beca
=> => exporting attestation manifest sha256:f8467946677a872e05755a1fab3be05e23e361e291a7c72c5dfc7a16fc6096d7
=> => exporting manifest list sha256:alice7d64801cf00cbc1f2e6d84be791243dcfb4a23150b880cbbe39dddeb2732
=> => naming to docker.io/library/flask-multistage:latest
=> => unpacking to docker.io/library/flask-multistage:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$

```

Step-22:- Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker images | grep flask-
flask-multistage      latest      a1ce7d64801c    25 hours ago    219MB
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple **** Dockerfile.multistage : ****

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
```

EXPOSE 5000

Run application

CMD ["python", "app.py"]

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker login
Authenticating with existing credentials... [Username: aarohi26]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

Step-24:- Build regular image

`docker build -t flask-regular .`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-25:- Build multi-stage image

`docker build -f Dockerfile.multistage -t flask-multistage .`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker push aarohi26/flaskapp:1.0
docker push aarohi26/flaskapp:latest
The push refers to repository [docker.io/aarohi26/flaskapp]
cc36eeef8762: Pushed
65d5eacb340e: Pushed
c7dfc3a6ec32: Pushed
38513bd72563: Pushed
b3ec39b36ae8: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
e144e59d3cce: Pushed
a8d60ef6ec86: Pushed
1.0: digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b size: 856
The push refers to repository [docker.io/aarohi26/flaskapp]
65d5eacb340e: Layer already exists
c7dfc3a6ec32: Layer already exists
a8d60ef6ec86: Layer already exists
ea56f685404a: Layer already exists
e144e59d3cce: Layer already exists
cc36eeef8762: Already exists
b3ec39b36ae8: Layer already exists
fc7443084902: Layer already exists
38513bd72563: Layer already exists
latest: digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b size: 856
```

Step-26:- Compare sizes

`docker images | grep flask-`

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker pull aaroHi26/flaskapp:latest
latest: Pulling from aaroHi26/flaskapp
Digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b
Status: Image is up to date for aaroHi26/flaskapp:latest
docker.io/aaroHi26/flaskapp:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker run -d -p 5000:5000 aaroHi26/flaskapp:latest
84832f1953727d5755d953c25b3f38d6545662a1d5c85995fa9126a6afd49765
```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ mkdir mynodeapp
cd mynodeapp
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano app.js
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
aarohi26@LAPTOP-4MMN5FCL x + v
GNU nano 7.2
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

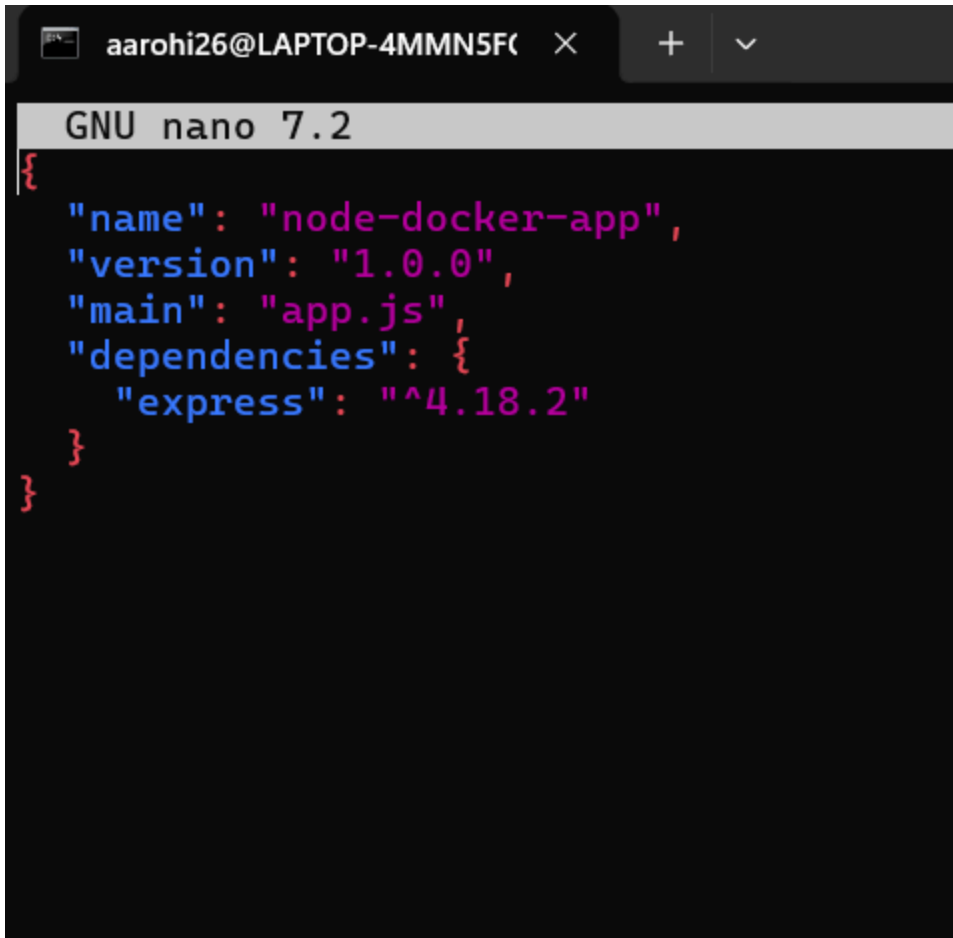
Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano package.json
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-31:- Pull from Docker Hub (on another machine)

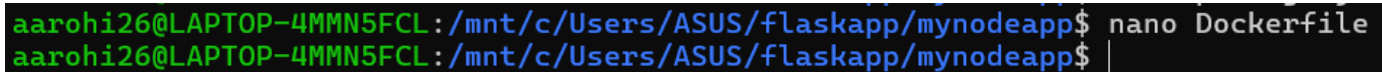
```
docker pull username/my-flask-app:latest
```



```
aarohi26@LAPTOP-4MMN5FC  X  +  v
GNU nano 7.2
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

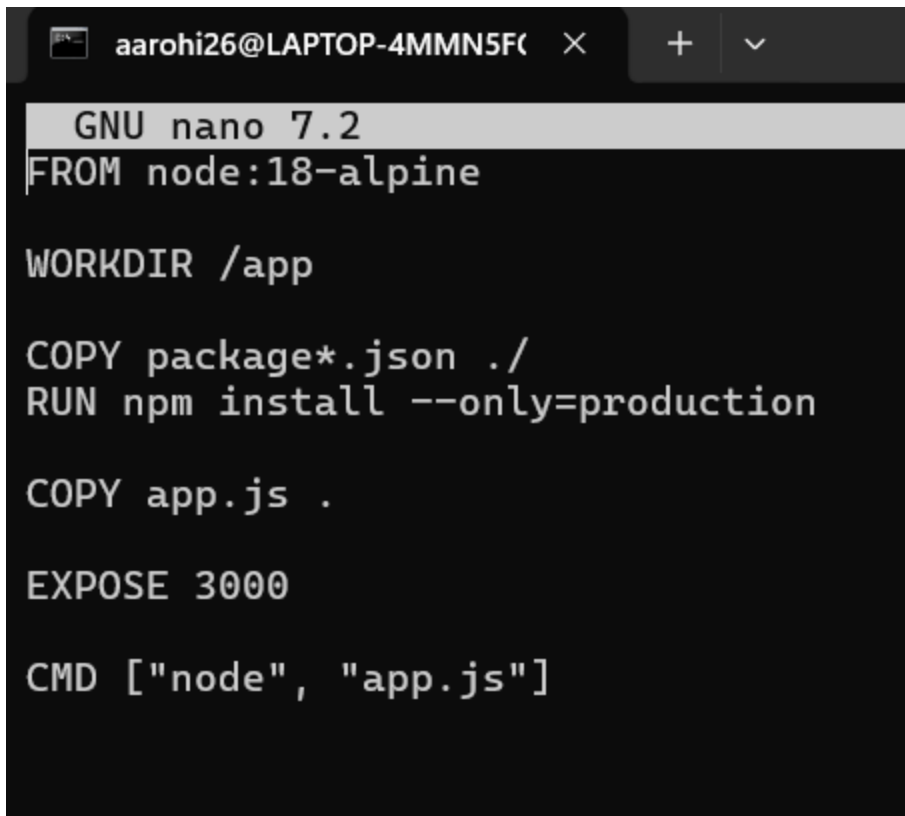


```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano Dockerfile
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app
cd my-node-app
```


A screenshot of a terminal window with a dark background. The title bar shows 'aaroji26@LAPTOP-4MMN5F...' and standard window controls. The terminal displays the GNU nano 7.2 editor interface. The text being edited is a Dockerfile with the following content: FROM node:18-alpine, WORKDIR /app, COPY package*.json ./, RUN npm install --only=production, COPY app.js ., EXPOSE 3000, and CMD ["node", "app.js"].

```
GNU nano 7.2
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step-34:- Create a **** app.js ****

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t mynodeapp .
[+] Building 20.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 514B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e 40.01MB / 40.01MB
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1a73b71a08505c6cc8a10e35ea87f562b902ff6eaa5890423faebe09d598ce36
=> => exporting config sha256:1d5339ee8e3e31d34e4480921b72606a8324cf71bb31d208f968eaa47f75bab
=> => exporting attestation manifest sha256:cbda2857cc39f5b4975ee93a51ca84eeb678b0c21ec3f2d11656818c3cbc1ea6
=> => exporting manifest list sha256:1a24a03a6ef47e3eca5e0d2b10befe5c3524e7ce43e4a95a9fbbe4594104af89
=> => naming to docker.io/library/mynodeapp:latest
=> => unpacking to docker.io/library/mynodeapp:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-35:- Create a **** package.json ****

```

{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker run -d -p 3001:3000 --name mynodecontainer mynodeapp
5375d72462d5ba43c642be39d0a5407bc4c9db31ebbf3ef06a20c6530670b50a
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-36:- Create a Node.js Dockerfile

```

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ curl http://localhost:3001
Hello from Node.js Docker!aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-37:- Build image

```
docker build -t my-node-app .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t myapp:latest -t myapp:v2.0 -t aarohi26/myapp:production .
[+] Building 18.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --only=production
=> CACHED [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1a73b71a08505c6cc8a10e35ea87f562b902ff6eaa5890423faebe09d598ce36
=> => exporting config sha256:1d5339ee8e3e31d34e4480921b72606a8324cf71bb31d208f968eeaa47f75bab
=> => exporting attestation manifest sha256:f1d0e519b386ecd6e7f0fc430e994912d57c5f74a6e3f8b79386c647757a57e6
=> => exporting manifest list sha256:bd03bd4fc08f26677db46fcc497c55b861a6d4dfdc1795dcb3dc7df696a77651
=> => naming to docker.io/library/myapp:latest
=> => unpacking to docker.io/library/myapp:latest
=> => naming to docker.io/library/myapp:v2.0
=> => unpacking to docker.io/library/myapp:v2.0
=> => naming to docker.io/aarohi26/myapp:production
=> => unpacking to docker.io/aarohi26/myapp:production
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-38:- Run container

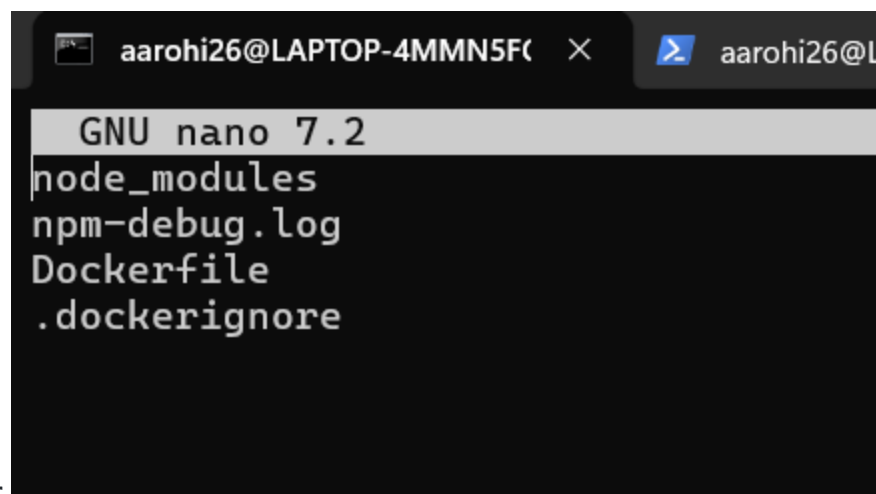
```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano Dockerfile2
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano .dockerignore
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```



Step-40:- Test on Browser

DockerHub

View Pushed Image on [DockerHub](#)

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build --no-cache -t clean-app .
[+] Building 7.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 93B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e8cc5b63d64bc1ddd1ffa5203ebb21bfa58cf44af39be6c4ceede3132fc108fc
=> => exporting config sha256:2c71dcc281f7fb7b39847e817ce1d6d6e723e46a13b87fe30bad1c00b919d575
=> => exporting attestation manifest sha256:7747b88e6c319382e3395be059332c9c05d84fef948411aee523e53f43e2153a
=> => exporting manifest list sha256:9e307f6a588267720353b86a37d96b44728644b3bdf150d22af7180f888b5da
=> => naming to docker.io/library/clean-app:latest
=> => unpacking to docker.io/library/clean-app:latest
=> => unpacking to docker.io/library/clean-app:latest
```

Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
docker build	Build image	docker build -t myapp .
docker run	Run container	docker run -p 3000:3000 myapp
docker ps	List containers	docker ps -a
docker images	List images	docker images
docker tag	Tag image	docker tag myapp:latest myapp:v1
docker login	Login to Dockerhub using username and password or token	echo "token" docker login -u username --password-stdin
docker push	Push to registry	docker push username/myapp
docker pull	Pull from registry	docker pull username/myapp

Command	Purpose	Example
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test Locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
2. **.dockerignore** excludes unnecessary files (important for security and performance)
3. **Tagging** helps version and organize images
4. **Multi-stage builds** create smaller, more secure production images
5. **Docker Hub** allows sharing and distributing images
6. **Always test** images locally before publishing

Cleanup

```
# Remove all stopped containers  
docker container prune
```

```
# Remove unused images  
docker image prune
```

```
# Remove everything unused  
docker system prune -a
```

[!\[\]\(10f8862fc183b400327470ea85afe9ae_img.jpg\) Previous](#) [!\[\]\(4ba8d838a2aa5445d51c9dee78fcb0cc_img.jpg\) Home](#)